YOUTUBE SHORTS
RECOMMENDATION SYSTEM
Complete Production Guide
VOLUME 1
Foundations, Architecture & Data Engineering
2025-2026 Edition

Latest Tech Stack: TensorFlow 2.18+ | PyTorch 2.5+ | TorchRec | FAISS 1.8+

Table of Contents

Chapter 1: Introduction to Recommendation Systems

1.1 What Are Recommendation Systems?

Recommendation systems are sophisticated software applications that predict and suggest items or content that users are likely to find relevant or interesting. These systems have become the backbone of modern digital platforms, powering everything from video streaming services to e-commerce websites and social media feeds.

At their core, recommendation systems solve a fundamental problem: information overload. With millions of products, videos, articles, and other content available online, users cannot possibly browse through everything to find what they want. Recommendation systems act as intelligent filters, surfacing the most relevant content based on user preferences, behavior, and contextual signals.

1.2 The Business Case for YouTube Shorts Recommendations

YouTube Shorts represents one of the most challenging and rewarding recommendation problems in the industry. Unlike traditional long-form video recommendations, Shorts operates in a rapid-consumption environment where users expect instant gratification and seamless content discovery.

The business objectives for a Shorts recommendation system include:

Maximizing User Engagement: The primary goal is to keep users actively engaged with the platform. This means recommending content that users will not only watch but also interact with through likes, comments, shares, and follows.

Increasing Session Duration: A well-tuned recommendation system extends the time users spend on the platform during each visit. For Shorts, this translates to users scrolling through more content and discovering new creators.

Supporting Creator Ecosystem: Recommendations must balance user preferences with fair exposure for creators. This ensures a healthy content ecosystem where new creators can gain visibility alongside established ones.

Revenue Optimization: While maintaining user experience, the system should create natural opportunities for monetization through ad placements and premium features.

1.3 Types of Recommendation Approaches

Understanding the fundamental approaches to building recommendation systems is essential before diving into implementation. Each approach has distinct strengths and is suited for different scenarios.

1.3.1 Collaborative Filtering

Collaborative filtering operates on the principle that users who have agreed in the past will agree in the future. This approach identifies patterns in user behavior to make predictions. There are two main variants:

User-Based Collaborative Filtering finds users similar to the target user based on their interaction history. If User A and User B both liked videos 1, 2, and 3, and User A also liked video 4, the system might recommend video 4 to User B.

Item-Based Collaborative Filtering focuses on finding relationships between items. If many users who watched Video A also watched Video B, these videos are considered similar. When a user watches Video A, the system recommends Video B.

Modern implementations use matrix factorization techniques to discover latent factors that explain observed interactions. These learned embeddings capture abstract concepts like genre preferences, mood associations, and content style without explicit feature engineering.

1.3.2 Content-Based Filtering

Content-based filtering recommends items similar to those a user has previously interacted with, based on item features. For video recommendations, these features might include video metadata like title, description, tags, duration, creator information, visual features extracted from thumbnails, audio characteristics, and engagement metrics.

This approach excels at handling the cold-start problem for new items since recommendations can be made based solely on content features. However, it may limit discovery by only recommending content similar to what users have already seen.

1.3.3 Hybrid Approaches

Production recommendation systems almost always use hybrid approaches that combine multiple techniques. A typical hybrid architecture might use content-based filtering for initial candidate generation, collaborative filtering for personalization scoring, and contextual bandits for exploration and handling uncertainty.

The YouTube Shorts system we will build uses a multi-stage hybrid approach with deep learning models that can learn complex feature interactions across all these paradigms simultaneously.

1.4 Key Metrics and Success Criteria

Measuring the success of a recommendation system requires a comprehensive set of metrics that capture both user satisfaction and business objectives.

Offline Metrics (Model Evaluation)

Metric	Description	Target Range
Recall@K	Proportion of relevant items in top K recommendations	>80% at K=1000
NDCG@K	Normalized discounted cumulative gain measuring ranking quality	>0.35 at K=10
MRR	Mean reciprocal rank of first relevant item	>0.25
Hit Rate	Percentage of users with at least one relevant recommendation	>90%

Online Metrics (Production Performance)

Metric	Description	Target
Watch Time	Average time spent watching recommended content	+15-25% improvement
CTR	Click-through rate on recommendation cards	>5%
Session Duration	Average length of user sessions	+10% improvement
Latency P99	99th percentile response time	<100ms

Chapter 2: System Architecture Overview

2.1 High-Level Architecture

The YouTube Shorts recommendation system follows a multi-stage architecture pattern that has become the industry standard for large-scale recommendation systems. This architecture balances computational efficiency with recommendation quality by progressively filtering and refining candidates through multiple stages.

The architecture consists of four primary stages:

Stage 1 - Candidate Generation: This stage rapidly retrieves a broad set of potentially relevant items from a corpus of millions of videos. The goal is high recall with acceptable precision, typically selecting 1,000-10,000 candidates from millions of possibilities in under 50 milliseconds.

Stage 2 - Ranking: A more sophisticated model scores and orders the candidates based on predicted user engagement. This stage can afford more computational complexity since it operates on a much smaller set of items.

Stage 3 - Re-ranking: Business rules, diversity constraints, and real-time signals are applied to adjust the final ordering. This stage ensures recommendations meet content policy requirements and provide variety.

Stage 4 - Serving: The final recommendations are delivered to users through an optimized serving infrastructure designed for low latency and high availability.

2.2 Candidate Generation Deep Dive

Candidate generation is arguably the most critical stage of the pipeline because errors here propagate through the entire system. If a relevant video is not retrieved as a candidate, it can never be recommended regardless of how good the ranking model is.

2.2.1 Two-Tower Architecture

The two-tower architecture has become the dominant approach for candidate generation in modern recommendation systems. It consists of two separate neural networks: a user tower that encodes user features into an embedding, and an item tower that encodes item features into an embedding of the same dimension.

During training, the towers learn to place users and items that should be matched close together in the embedding space, while pushing non-matching pairs apart. The training objective typically uses a contrastive loss function like sampled softmax or in-batch negatives.

The key advantage of this architecture is inference efficiency. Once trained, item embeddings can be pre-computed and indexed. At serving time, only the user tower needs to run to produce a query embedding, which is then used to retrieve similar items through approximate nearest neighbor search.

2.2.2 Approximate Nearest Neighbor (ANN) Search

Finding the most similar items in a high-dimensional embedding space is computationally expensive. For a corpus of millions of items, exact nearest neighbor search is prohibitively slow. Approximate nearest neighbor algorithms trade a small amount of accuracy for dramatic speedups.

The two leading libraries for ANN search in 2025-2026 are FAISS (developed by Meta) and ScaNN (developed by Google). FAISS excels with its GPU acceleration capabilities and broad set of indexing algorithms, making it ideal for large-scale scenarios. ScaNN specializes in maximum inner product search (MIPS) with its anisotropic vector quantization technique, often achieving higher accuracy for recommendation-specific similarity metrics.

A typical configuration uses FAISS with an IVF (Inverted File Index) structure combined with Product Quantization (PQ) for memory efficiency. This allows searching through tens of millions of embeddings in under 10 milliseconds while using a fraction of the memory required for brute-force search.

2.3 Ranking Model Architecture

The ranking model receives candidate items from the generation stage and produces a relevance score for each item. Unlike candidate generation which prioritizes speed, the ranking model can use more complex architectures since it operates on thousands rather than millions of items.

2.3.1 Deep Cross Network (DCN)

Deep Cross Networks explicitly model feature interactions at multiple orders while maintaining computational efficiency. The cross network applies feature crossing at each layer, automatically learning important feature combinations without manual feature engineering.

The architecture combines a cross network for explicit interactions with a deep network for implicit interactions. The cross network efficiently captures polynomial feature interactions, while the deep network learns arbitrary nonlinear transformations. The outputs are combined for final prediction.

2.3.2 Transformer-Based Ranking

Transformer architectures have revolutionized ranking models by their ability to capture sequential patterns in user behavior. For Shorts recommendations, transformers can model the sequence of recently watched videos to understand user intent and predict what they want to see next.

The self-attention mechanism allows the model to weigh the importance of different items in the user's history dynamically. A video watched three interactions ago might be more predictive than the immediately previous one if it signals a shift in user interest.

2.4 Multi-Objective Optimization

Real recommendation systems must optimize for multiple objectives simultaneously. For Shorts, these objectives include watch time (how long users watch each video), engagement (likes, comments, shares), completion rate (percentage of video watched), creator diversity (ensuring varied content sources), and freshness (balancing new content with proven performers).

These objectives often conflict. Clickbait videos might maximize short-term engagement but hurt long-term user satisfaction. The system must find a balance through multi-objective optimization techniques.

Common approaches include scalarization (combining objectives into a single weighted score), Pareto optimization (finding solutions that cannot improve one objective without hurting another), and constrained optimization (maximizing primary objective subject to constraints on secondary objectives).

Chapter 3: Technology Stack Deep Dive (2025-2026)

3.1 Overview of Modern ML Infrastructure

Building a production recommendation system requires a carefully selected technology stack that balances performance, scalability, developer productivity, and operational maintainability. The 2025-2026 landscape offers mature, battle-tested tools alongside innovative new solutions.

Our reference architecture prioritizes open-source technologies with strong community support, cloud-native design patterns, and proven production deployments at scale. This ensures you can adapt the system to various deployment environments while benefiting from continuous improvements.

3.2 Deep Learning Frameworks

3.2.1 TensorFlow 2.18+ and TensorFlow Recommenders

TensorFlow remains a top choice for recommendation systems due to its comprehensive ecosystem and production-ready tooling. TensorFlow Recommenders (TFRS) provides high-level APIs specifically designed for building retrieval and ranking models.

Key features of TensorFlow Recommenders include built-in retrieval tasks with contrastive loss functions, factorized top-K layers for efficient candidate retrieval, seamless integration with TensorFlow Serving for production deployment, and support for multi-task learning with shared embeddings.

Example: Basic Two-Tower Model with TFRS

```
import tensorflow as tf

import tensorflow_recommenders as tfrs



class TwoTowerModel(tfrs.Model):

    def __init__(self, user_model, item_model, task):

        super().__init__()

        self.user_model = user_model
```

```
        self.item_model = item_model
        self.task = task

    def compute_loss(self, features, training=False):
        user_embeddings = self.user_model(features['user_id'])
        item_embeddings = self.item_model(features['video_id'])
        return self.task(user_embeddings, item_embeddings)

# Create user tower
user_model = tf.keras.Sequential([
    tf.keras.layers.StringLookup(vocabulary=user_ids),
    tf.keras.layers.Embedding(len(user_ids) + 1, 64)
])

# Create retrieval task with metrics
task = tfrs.tasks.Retrieval(
    metrics=tfrs.metrics.FactorizedTopK(
        candidates=videos.batch(128).map(item_model)
    )
)
```

3.2.2 PyTorch 2.5+ and TorchRec

PyTorch has become the framework of choice for research and increasingly for production systems. TorchRec, Meta's domain library for recommendation systems, brings production-grade capabilities for large-scale embedding operations.

TorchRec provides native support for embedding table sharding across multiple GPUs, optimized sparse operations through FBGEMM kernels, flexible parallelism strategies including model and data parallelism, and seamless integration with PyTorch's ecosystem for experimentation.

Example: TorchRec Embedding Collection

```
import torch

from torchrec import EmbeddingBagCollection

from torchrec.sparse.jagged_tensor import KeyedJaggedTensor



# Define embedding tables

embedding_configs = [

    EmbeddingBagConfig(

        name='user_embedding',

        embedding_dim=64,

        num_embeddings=1_000_000,

        feature_names=['user_id']

    ),

    EmbeddingBagConfig(

        name='video_embedding',

        embedding_dim=64,

        num_embeddings=10_000_000,

        feature_names=['video_id']

    )
```

```
]



# Create sharded embedding collection
ebc = EmbeddingBagCollection(
    tables=embedding_configs,
    device=torch.device('cuda')
)
```

3.3 Vector Search and ANN Libraries

3.3.1 FAISS 1.8+

FAISS (Facebook AI Similarity Search) is the industry standard for efficient similarity search. Version 1.8+ includes significant improvements in GPU utilization, new indexing algorithms, and better support for billion-scale datasets.

Index Type	Use Case	Memory vs Speed
Flat	Exact search, small datasets (<100K)	High memory, fastest for small data
IVF	Medium datasets, good balance	Configurable via nlist/nprobe
IVF-PQ	Large datasets, memory constrained	Low memory, slight accuracy loss
HNSW	Highest accuracy requirements	Higher memory, excellent recall

Example: FAISS Index Construction

```
import faiss
import numpy as np



# Configuration
```

```
dimension = 64
num_vectors = 10_000_000
nlist = 4096  # Number of clusters
m = 16        # Number of subquantizers

# Create IVF-PQ index for memory efficiency
quantizer = faiss.IndexFlatIP(dimension)
index = faiss.IndexIVFPQ(
    quantizer, dimension, nlist, m, 8
)

# Train on representative sample
training_vectors = np.random.randn(100_000, dimension).astype('float32')
faiss.normalize_L2(training_vectors)
index.train(training_vectors)

# Add vectors
vectors = np.random.randn(num_vectors, dimension).astype('float32')
faiss.normalize_L2(vectors)
index.add(vectors)

# Search
index.nprobe = 64  # Trade-off: higher = more accurate, slower
```

```
query = np.random.randn(1, dimension).astype('float32')

faiss.normalize_L2(query)

distances, indices = index.search(query, k=100)
```

3.3.2 ScaNN 1.3+

ScaNN (Scalable Nearest Neighbors) from Google excels at maximum inner product search (MIPS), which is the natural similarity metric for recommendation embeddings. Its anisotropic vector quantization technique preserves ranking accuracy better than traditional quantization methods.

ScaNN is particularly effective when integrated with TensorFlow pipelines and provides optimized performance for the types of embedding similarity searches common in recommendation systems.

3.4 Data Processing and Feature Stores

3.4.1 Apache Spark 3.5+ with Delta Lake

Apache Spark remains the backbone of large-scale data processing. Combined with Delta Lake for ACID transactions and time travel capabilities, it provides a robust foundation for feature engineering pipelines.

3.4.2 Polars for Fast DataFrame Operations

Polars has emerged as a high-performance alternative to Pandas for data manipulation. Written in Rust, it offers significantly faster execution for most operations while maintaining a familiar API. For recommendation system development, Polars excels at feature engineering on medium-scale datasets that fit in memory.

Example: Feature Engineering with Polars

```
import polars as pl



# Load interaction data

interactions = pl.read_parquet('interactions.parquet')
```

```
# Compute user-level features
user_features = interactions.group_by('user_id').agg([
    pl.col('watch_time').sum().alias('total_watch_time'),
    pl.col('video_id').n_unique().alias('unique_videos'),
    pl.col('liked').sum().alias('total_likes'),
    pl.col('timestamp').max().alias('last_active'),
    pl.col('category').mode().first().alias('favorite_category')
])

# Compute engagement rate
user_features = user_features.with_columns([
    (pl.col('total_likes') / pl.col('unique_videos'))
    .alias('avg_engagement_rate')
])
```

3.4.3 Feature Stores: Feast and Hopsworks

Feature stores solve the critical problem of serving consistent features between training and inference. Feast provides an open-source option with support for both batch and real-time features, while Hopsworks offers a more comprehensive managed solution.

Key capabilities include point-in-time correct feature retrieval (preventing data leakage), low-latency online serving backed by Redis or similar stores, feature versioning and lineage tracking, and integration with ML pipelines for automated feature refresh.

3.5 Model Serving Infrastructure

3.5.1 TensorFlow Serving

TensorFlow Serving provides production-grade model serving with features like A/B testing support, model versioning, and automatic batching for GPU efficiency. It integrates naturally with TensorFlow models but can also serve models from other frameworks through its generic prediction API.

3.5.2 Triton Inference Server

NVIDIA Triton has become the go-to solution for GPU-optimized model serving. It supports multiple frameworks (TensorFlow, PyTorch, ONNX), provides sophisticated batching strategies, and offers concurrent model execution to maximize GPU utilization.

3.5.3 Deployment Patterns

Modern recommendation serving typically uses a combination of approaches. Embedding lookup services pre-compute and cache item embeddings with Redis or similar for sub-millisecond retrieval. Real-time inference services run user tower and ranking models on each request. Sidecar patterns deploy lightweight re-ranking logic close to the application.

3.6 MLOps and Pipeline Orchestration

Tool	Primary Use	Key Features
Kubeflow	End-to-end ML pipelines on Kubernetes	Scalable, cloud-native, component reuse
MLflow 2.x	Experiment tracking, model registry	Language agnostic, easy integration
Weights & Biases	Experiment tracking, visualization	Rich UI, collaborative features
Dagster	Data orchestration, asset management	Software-defined assets, observability
Ray	Distributed computing, hyperparameter tuning	Unified API, scales from laptop to cluster

Chapter 4: Data Engineering & Schema Design

4.1 Understanding the Data Landscape

A recommendation system is only as good as its data. This chapter covers the fundamental data entities, their relationships, and best practices for schema design that supports both model training and real-time serving.

The core data entities in a Shorts recommendation system include users (people who consume content), items (the short-form videos), interactions (events capturing user-item relationships), and context (situational information about when and how interactions occur).

4.2 User Data Schema

User data captures both static profile information and dynamic behavioral signals. The schema must support efficient lookups while maintaining flexibility for feature evolution.

User Profile Schema (Static)

```
CREATE TABLE user_profiles (

    user_id         STRING PRIMARY KEY,

    created_at      TIMESTAMP,

    country_code    STRING,

    language        STRING,

    age_bucket      STRING,  -- '18-24', '25-34', etc.

    device_type     STRING,  -- 'ios', 'android', 'web'

    account_type    STRING,  -- 'free', 'premium'

    creator_flag    BOOLEAN, -- Is user also a creator?

    updated_at      TIMESTAMP

);



-- Partitioned by country for query efficiency
```

```
PARTITION BY (country_code)
```

User Behavioral Features (Dynamic)

```
CREATE TABLE user_features (
    user_id                 STRING,
    feature_date            DATE,

    -- Engagement metrics (last 7 days)
    videos_watched_7d       INT64,
    total_watch_time_7d     FLOAT64,
    avg_completion_rate_7d  FLOAT64,
    likes_given_7d          INT64,
    shares_7d               INT64,
    comments_7d             INT64,

    -- Content preferences
    top_categories          ARRAY<STRING>,
    top_creators_followed   ARRAY<STRING>,
    preferred_video_length  STRING,  -- 'short', 'medium', 'long'

    -- Session patterns
    avg_session_length      FLOAT64,
    sessions_per_day        FLOAT64,
    peak_activity_hour      INT64,
```

```
    -- Embedding (pre-computed)
    user_embedding          ARRAY<FLOAT64>,

    PRIMARY KEY (user_id, feature_date)
);
```

4.3 Video Data Schema

Video data combines creator-provided metadata with system-computed features. The schema supports both content-based filtering and popularity-based signals.

Video Metadata Schema

```
CREATE TABLE videos (
    video_id            STRING PRIMARY KEY,
    creator_id          STRING,
    upload_timestamp    TIMESTAMP,

    -- Content metadata
    duration_seconds    FLOAT64,
    title               STRING,
    description         STRING,
    tags                ARRAY<STRING>,
    category            STRING,
    language            STRING,
```

```
    -- Technical attributes
    resolution          STRING,
    has_audio           BOOLEAN,
    has_music           BOOLEAN,
    music_id            STRING,  -- If uses licensed music

    -- Content signals (ML-derived)
    thumbnail_embedding ARRAY<FLOAT64>,
    audio_embedding     ARRAY<FLOAT64>,
    text_embedding      ARRAY<FLOAT64>,  -- From title/description
    content_embedding   ARRAY<FLOAT64>,  -- Combined representation

    -- Moderation
    is_approved         BOOLEAN,
    content_rating      STRING,  -- 'G', 'PG', 'PG-13', etc.

    updated_at          TIMESTAMP
);

-- Index for category-based queries
CREATE INDEX idx_videos_category ON videos(category);
```

Video Engagement Statistics (Time-Series)

```
CREATE TABLE video_stats (
```

```
    video_id            STRING,
    stat_date           DATE,
    stat_hour           INT64,  -- 0-23 for hourly granularity

    -- View metrics
    view_count          INT64,
    unique_viewers      INT64,
    total_watch_time    FLOAT64,
    avg_watch_percentage FLOAT64,

    -- Engagement metrics
    like_count          INT64,
    dislike_count       INT64,
    comment_count       INT64,
    share_count         INT64,

    -- Quality signals
    skip_rate           FLOAT64,  -- % who skip within 2 seconds
    rewatch_rate        FLOAT64,  -- % who watch multiple times
    follow_rate         FLOAT64,  -- % who follow creator after

    PRIMARY KEY (video_id, stat_date, stat_hour)
);
```

4.4 Interaction Data Schema

Interaction data is the lifeblood of the recommendation system. It captures every meaningful event between users and videos, forming the training signal for models.

Interaction Events Schema

```
CREATE TABLE interactions (

    interaction_id      STRING PRIMARY KEY,

    user_id             STRING,

    video_id            STRING,

    timestamp           TIMESTAMP,


    -- Interaction type

    event_type          STRING,  -- 'impression', 'view', 'complete',

                                 -- 'like', 'share', 'comment', 'skip'


    -- View details (if applicable)

    watch_duration      FLOAT64,

    watch_percentage    FLOAT64,

    rewatched           BOOLEAN,


    -- Context

    source              STRING,  -- 'feed', 'search', 'profile', 'share'

    position_in_feed    INT64,

    session_id          STRING,

    device_type         STRING,
```

```
    -- Recommendation context
    recommendation_id   STRING,  -- Links to which rec served this
    model_version       STRING,
    candidate_score     FLOAT64  -- Score assigned by ranking model
);

-- Partition by date for efficient historical queries
PARTITION BY DATE(timestamp)

-- Cluster by user_id for user-centric queries
CLUSTER BY user_id
```

4.5 Data Quality and Validation

Data quality issues can silently degrade recommendation quality. Implement validation at multiple stages of the pipeline.

Schema Validation with Great Expectations

```
import great_expectations as gx

# Create expectation suite for interactions
suite = gx.ExpectationSuite('interactions_quality')

# Required fields must be present
```

```
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToNotBeNull(
        column='user_id'
    )
)
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToNotBeNull(
        column='video_id'
    )
)


# Watch percentage should be valid
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeBetween(
        column='watch_percentage',
        min_value=0.0,
        max_value=1.0
    )
)


# Event types should be from known set
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeInSet(
        column='event_type',
```

```
        value_set=['impression', 'view', 'complete',
                   'like', 'share', 'comment', 'skip']
    )
)
```

Chapter 5: Data Ingestion & Processing Pipelines

5.1 Pipeline Architecture Overview

Data pipelines in a recommendation system must handle multiple data sources, process at different time scales, and maintain consistency between batch and streaming paths. This chapter details the implementation of robust, scalable pipelines.

The architecture follows the Lambda pattern, maintaining both batch and speed layers. The batch layer processes historical data for model training, while the speed layer handles real-time features for serving. A serving layer combines outputs from both for low-latency recommendations.

5.2 Batch Processing Pipeline

The batch pipeline runs on a daily schedule, processing the previous day's interactions to update user features, video statistics, and training datasets.

Daily Feature Pipeline with Dagster

```
from dagster import asset, Definitions, ScheduleDefinition

import polars as pl

from datetime import datetime, timedelta



@asset(description='Raw interactions from event stream')

def raw_interactions(context):

    """Load raw interaction events from data lake."""

    yesterday = datetime.now() - timedelta(days=1)

    path = f's3://data-lake/interactions/{yesterday:%Y/%m/%d}/'



    df = pl.read_parquet(path)

    context.log.info(f'Loaded {len(df):,} interactions')

    return df
```

```
@asset(description='Validated and cleaned interactions')
def validated_interactions(raw_interactions):
    """Apply data quality checks and cleaning."""
    df = raw_interactions

    # Remove invalid records
    df = df.filter(
        pl.col('user_id').is_not_null() &
        pl.col('video_id').is_not_null() &
        pl.col('watch_percentage').is_between(0, 1)
    )

    # Deduplicate based on interaction_id
    df = df.unique(subset=['interaction_id'])

    return df


@asset(description='Aggregated user features')
def user_features_daily(validated_interactions):
    """Compute daily user feature aggregations."""
    return validated_interactions.group_by('user_id').agg([
        pl.len().alias('interactions_count'),
        pl.col('watch_duration').sum().alias('total_watch_time'),
```

```
        pl.col('watch_percentage').mean().alias('avg_completion'),
        pl.col('event_type').filter(
            pl.col('event_type') == 'like'
        ).len().alias('likes_given'),
        pl.col('category').mode().first().alias('top_category')
    ])
```

5.3 Streaming Pipeline for Real-Time Features

Real-time features capture recent user behavior that is too fresh for batch processing. These features are critical for session-based personalization.

Real-Time Feature Computation with Apache Flink (Python)

```
from pyflink.datastream import StreamExecutionEnvironment
from pyflink.datastream.connectors.kafka import KafkaSource
from pyflink.common.serialization import SimpleStringSchema
from pyflink.datastream.window import TumblingEventTimeWindows
from pyflink.common.time import Time


# Create streaming environment
env = StreamExecutionEnvironment.get_execution_environment()


# Configure Kafka source for interaction events
kafka_source = KafkaSource.builder() \
    .set_bootstrap_servers('kafka:9092') \
    .set_topics('user-interactions') \
```

```
    .set_group_id('realtime-features') \
    .set_value_only_deserializer(SimpleStringSchema()) \
    .build()


# Create data stream
stream = env.from_source(
    kafka_source,
    WatermarkStrategy.for_bounded_out_of_orderness(
        Duration.of_seconds(30)
    ),
    'Kafka Interactions'
)


# Compute rolling window features
user_session_features = stream \
    .key_by(lambda x: x['user_id']) \
    .window(TumblingEventTimeWindows.of(Time.minutes(5))) \
    .aggregate(SessionFeatureAggregator())
```

5.4 Feature Store Integration

The feature store serves as the bridge between feature pipelines and model serving, ensuring consistent features across training and inference.

Feast Feature Store Configuration

```
# feature_store.yaml
```

```
project: shorts_recommendations

provider: gcp

registry: gs://feast-registry/registry.db

online_store:

  type: redis

  connection_string: redis://redis-cluster:6379

offline_store:

  type: bigquery

entity_key_serialization_version: 2
```

Feature Definitions

```
from feast import Entity, Feature, FeatureView, ValueType

from feast.data_sources import BigQuerySource

from datetime import timedelta


# Define entities

user = Entity(

    name='user_id',

    value_type=ValueType.STRING,

    description='Unique user identifier'

)


video = Entity(
```

```
    name='video_id',
    value_type=ValueType.STRING,
    description='Unique video identifier'
)

# Define user features
user_features_source = BigQuerySource(
    table='project.dataset.user_features',
    timestamp_field='feature_date'
)

user_features_fv = FeatureView(
    name='user_features',
    entities=['user_id'],
    ttl=timedelta(days=7),
    features=[
        Feature(name='videos_watched_7d', dtype=ValueType.INT64),
        Feature(name='total_watch_time_7d', dtype=ValueType.FLOAT),
        Feature(name='avg_completion_rate_7d', dtype=ValueType.FLOAT),
        Feature(name='top_categories', dtype=ValueType.STRING_LIST),
        Feature(name='user_embedding', dtype=ValueType.FLOAT_LIST),
    ],
    source=user_features_source
)
```

5.5 Training Data Generation

Generating high-quality training data requires careful consideration of sampling strategies, label definition, and temporal consistency.

Training Dataset Pipeline

```
import polars as pl

from datetime import datetime, timedelta



def generate_training_data(

    interactions_path: str,

    features_path: str,

    start_date: datetime,

    end_date: datetime

) -> pl.DataFrame:

    """

    Generate training data with point-in-time correct features.

    """

    # Load interactions

    interactions = pl.scan_parquet(interactions_path).filter(

        pl.col('timestamp').is_between(start_date, end_date)

    )



    # Create positive examples (watched > 50% or engaged)

    positives = interactions.filter(
```

```
        (pl.col('watch_percentage') > 0.5) |
        (pl.col('event_type').is_in(['like', 'share', 'comment']))
    ).with_columns(pl.lit(1).alias('label'))

    # Create negative examples (skipped quickly)
    negatives = interactions.filter(
        (pl.col('watch_percentage') < 0.1) &
        (pl.col('event_type') == 'skip')
    ).with_columns(pl.lit(0).alias('label'))

    # Combine and join with features
    training = pl.concat([positives, negatives])

    # Point-in-time join with user features
    # (features must be from BEFORE the interaction)
    user_features = pl.scan_parquet(features_path)

    training = training.join_asof(
        user_features,
        left_on='timestamp',
        right_on='feature_date',
        by='user_id',
        strategy='backward'  # Use features from before interaction
    )
```

```
    return training.collect()
```

Chapter 6: Feature Engineering Fundamentals

6.1 The Art and Science of Feature Engineering

Feature engineering transforms raw data into informative signals that machine learning models can leverage effectively. In recommendation systems, good features capture user preferences, item characteristics, and contextual relevance.

Features can be categorized along several dimensions: static versus dynamic (how frequently they change), dense versus sparse (cardinality and representation), and user-side versus item-side versus contextual (what entity they describe).

6.2 User Feature Categories

6.2.1 Demographic Features

Demographic features provide baseline information about users but should be used carefully to avoid reinforcing biases. These include age group, geographic region, language preferences, and device type. These features help with cold-start scenarios and can inform content localization.

6.2.2 Behavioral Features

Behavioral features capture how users interact with the platform over time. They are typically more predictive than demographic features. Key categories include engagement intensity (how active the user is overall), content preferences (what types of content they consume), temporal patterns (when they are most active), and interaction quality (how deeply they engage with content).

Computing Behavioral Features

```
def compute_user_behavioral_features(
    interactions: pl.DataFrame,
    lookback_days: int = 30
) -> pl.DataFrame:
    """
    Compute comprehensive behavioral features for users.
    """
```

```
    cutoff = datetime.now() - timedelta(days=lookback_days)

    recent = interactions.filter(pl.col('timestamp') > cutoff)

    features = recent.group_by('user_id').agg([
        # Engagement intensity
        pl.len().alias('total_interactions'),
        pl.col('session_id').n_unique().alias('session_count'),
        (pl.len() / lookback_days).alias('daily_activity'),

        # Watch behavior
        pl.col('watch_duration').sum().alias('total_watch_time'),
        pl.col('watch_duration').mean().alias('avg_watch_duration'),
        pl.col('watch_percentage').mean().alias('avg_completion_rate'),
        pl.col('watch_percentage').std().alias('completion_variance'),

        # Engagement depth
        pl.col('event_type').filter(
            pl.col('event_type') == 'like'
        ).len().alias('like_count'),
        pl.col('event_type').filter(
            pl.col('event_type') == 'share'
        ).len().alias('share_count'),
        pl.col('event_type').filter(
```

```
            pl.col('event_type') == 'comment'
        ).len().alias('comment_count'),

        # Content preferences
        pl.col('category').mode().first().alias('favorite_category'),
        pl.col('category').n_unique().alias('category_diversity'),
        pl.col('creator_id').n_unique().alias('creators_watched'),

        # Temporal patterns
        pl.col('timestamp').dt.hour().mode().first()
            .alias('peak_hour'),
        pl.col('timestamp').dt.weekday().mode().first()
            .alias('peak_day'),
    ])

    # Compute derived ratios
    features = features.with_columns([
        (pl.col('like_count') / pl.col('total_interactions'))
            .alias('like_rate'),
        (pl.col('share_count') / pl.col('total_interactions'))
            .alias('share_rate'),
        (pl.col('creators_watched') / pl.col('total_interactions'))
            .alias('exploration_rate'),
    ])
```

```
    return features
```

6.3 Video Feature Categories

6.3.1 Content Features

Content features describe what the video is about and how it is presented. These include metadata features from creator input, visual features extracted from frames, audio features from the soundtrack, and text features from titles and descriptions.

6.3.2 Engagement Features

Engagement features capture how the audience has responded to the video. These are powerful predictors but must be used carefully to avoid popularity bias. Key metrics include view-through rate, engagement rate, freshness decay, and velocity metrics.

Video Feature Computation

```
def compute_video_features(
    video_stats: pl.DataFrame,
    videos: pl.DataFrame,
    current_time: datetime
) -> pl.DataFrame:
    """
    Compute video features combining content and engagement signals.
    """
    # Recent engagement (last 7 days)
    recent_cutoff = current_time - timedelta(days=7)
    recent_stats = video_stats.filter(
        pl.col('stat_date') > recent_cutoff
```

```
)

engagement_features = recent_stats.group_by('video_id').agg([
    pl.col('view_count').sum().alias('views_7d'),
    pl.col('like_count').sum().alias('likes_7d'),
    pl.col('share_count').sum().alias('shares_7d'),
    pl.col('avg_watch_percentage').mean()
        .alias('avg_completion_7d'),
    pl.col('skip_rate').mean().alias('avg_skip_rate'),
])

# Compute engagement rates
engagement_features = engagement_features.with_columns([
    (pl.col('likes_7d') / pl.col('views_7d').clip(1, None))
        .alias('like_rate'),
    (pl.col('shares_7d') / pl.col('views_7d').clip(1, None))
        .alias('share_rate'),
])

# Join with video metadata
features = videos.join(
    engagement_features,
    on='video_id',
    how='left'
```

```
    )


    # Compute freshness
    features = features.with_columns([
        ((current_time - pl.col('upload_timestamp'))
            .dt.total_hours() / 24).alias('age_days'),
    ])


    # Freshness decay (exponential)
    features = features.with_columns([
        (0.95 ** pl.col('age_days')).alias('freshness_score'),
    ])


    return features
```

6.4 Cross Features and Interactions

Some of the most powerful signals come from interactions between user and item features. These cross features capture compatibility and contextual relevance.

Cross Feature Examples

```
def compute_cross_features(
    user_features: pl.DataFrame,
    video_features: pl.DataFrame,
    user_id: str,
    video_id: str
```

```
) -> dict:
    """
    Compute user-item cross features for ranking.
    """
    user = user_features.filter(
        pl.col('user_id') == user_id
    ).to_dicts()[0]

    video = video_features.filter(
        pl.col('video_id') == video_id
    ).to_dicts()[0]

    cross = {}

    # Category match
    cross['category_match'] = int(
        video['category'] in user.get('top_categories', [])
    )

    # Creator familiarity
    cross['creator_followed'] = int(
        video['creator_id'] in user.get('followed_creators', [])
    )
```

```
    # Language match
    cross['language_match'] = int(
        video['language'] == user.get('language', '')
    )

    # Duration preference match
    user_preferred_duration = user.get('avg_watch_duration', 30)
    video_duration = video.get('duration_seconds', 30)
    cross['duration_fit'] = 1.0 - min(
        abs(video_duration - user_preferred_duration) / 60,
        1.0
    )

    # Time-of-day relevance
    current_hour = datetime.now().hour
    user_peak_hour = user.get('peak_hour', 12)
    hour_diff = min(
        abs(current_hour - user_peak_hour),
        24 - abs(current_hour - user_peak_hour)
    )
    cross['time_relevance'] = 1.0 - (hour_diff / 12)

    return cross
```

6.5 Feature Normalization and Encoding

Proper feature preprocessing is essential for model training stability and performance.

Feature Preprocessing Pipeline

```
import numpy as np

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.compose import ColumnTransformer



class FeaturePreprocessor:

    def __init__(self):

        self.numeric_scaler = StandardScaler()

        self.categorical_encoders = {}



    def fit(self, df: pl.DataFrame, config: dict):

        """Fit preprocessors on training data."""

        # Fit numeric scalers

        numeric_cols = config['numeric_features']

        self.numeric_scaler.fit(

            df.select(numeric_cols).to_numpy()

        )



        # Fit categorical encoders

        for col in config['categorical_features']:

            encoder = LabelEncoder()

            # Add unknown category for unseen values
```

```
            values = df[col].to_list() + ['__unknown__']
            encoder.fit(values)
            self.categorical_encoders[col] = encoder

    def transform(self, df: pl.DataFrame, config: dict):
        """Transform features for model input."""
        result = df.clone()

        # Scale numeric features
        numeric_cols = config['numeric_features']
        scaled = self.numeric_scaler.transform(
            result.select(numeric_cols).to_numpy()
        )
        for i, col in enumerate(numeric_cols):
            result = result.with_columns(
                pl.lit(scaled[:, i]).alias(col)
            )

        # Encode categorical features
        for col in config['categorical_features']:
            encoder = self.categorical_encoders[col]
            # Handle unseen categories
            def safe_encode(x):
                try:
```

```
                return encoder.transform([x])[0]
            except ValueError:
                return encoder.transform(['__unknown__'])[0]

        result = result.with_columns(
            pl.col(col).map_elements(safe_encode).alias(col)
        )

    return result
```

Appendix A: Environment Setup Guide

A.1 Development Environment Requirements

Component	Minimum	Recommended
Python	3.10	3.11 or 3.12
RAM	16 GB	32 GB+
GPU	NVIDIA RTX 3080	NVIDIA A100 / RTX 4090
CUDA	12.1	12.4+
Storage	100 GB SSD	500 GB+ NVMe

A.2 Python Environment Setup

Environment Setup Commands

```
# Create conda environment
conda create -n shorts-recsys python=3.11 -y
conda activate shorts-recsys

# Install PyTorch with CUDA support
pip install torch==2.5.0 torchvision torchaudio \
    --index-url https://download.pytorch.org/whl/cu124

# Install TensorFlow
pip install tensorflow==2.18.0 tensorflow-recommenders

# Install TorchRec (Meta's recommendation library)
```

```
pip install fbgemm-gpu --index-url https://download.pytorch.org/whl/cu124

pip install torchrec --index-url https://download.pytorch.org/whl/cu124



# Install data processing libraries

pip install polars==1.0.0 pandas numpy scipy



# Install FAISS with GPU support

pip install faiss-gpu==1.8.0



# Install MLOps tools

pip install mlflow==2.16.0 wandb dagster feast



# Install additional utilities

pip install scikit-learn great-expectations pydantic



# Verify GPU availability

python -c "import torch; print(f'CUDA available:
{torch.cuda.is_available()}')"
```

A.3 Docker Development Environment

Dockerfile for Development

```
FROM nvidia/cuda:12.4.0-devel-ubuntu22.04
```

```
# Set environment variables
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV DEBIAN_FRONTEND=noninteractive

# Install system dependencies
RUN apt-get update && apt-get install -y \
    python3.11 python3.11-dev python3-pip \
    git curl wget && \
    rm -rf /var/lib/apt/lists/*

# Set Python 3.11 as default
RUN update-alternatives --install /usr/bin/python python \
    /usr/bin/python3.11 1

# Install Python packages
COPY requirements.txt /tmp/
RUN pip install --no-cache-dir -r /tmp/requirements.txt

# Set working directory
WORKDIR /workspace

# Default command
```

```
CMD ["/bin/bash"]
```

Appendix B: Code Repository Structure

B.1 Project Layout

```
shorts-recsys/
|-- README.md
|-- pyproject.toml
|-- requirements.txt
|-- Dockerfile
|-- docker-compose.yml
|
|-- config/
|   |-- model_config.yaml
|   |-- feature_config.yaml
|   |-- serving_config.yaml
|
|-- src/
|   |-- __init__.py
|   |
|   |-- data/
|   |   |-- __init__.py
|   |   |-- loaders.py          # Data loading utilities
|   |   |-- schemas.py          # Pydantic schemas
|   |   |-- validation.py       # Data quality checks
|   |
|   |-- features/
```

```
|   |   |-- __init__.py
|   |   |-- user_features.py      # User feature computation
|   |   |-- video_features.py     # Video feature computation
|   |   |-- cross_features.py     # Cross features
|   |   |-- preprocessing.py      # Feature preprocessing
|   |
|   |-- models/
|   |   |-- __init__.py
|   |   |-- two_tower.py          # Two-tower retrieval model
|   |   |-- ranking.py            # Ranking model
|   |   |-- embeddings.py         # Embedding layers
|   |
|   |-- training/
|   |   |-- __init__.py
|   |   |-- trainer.py            # Training loop
|   |   |-- losses.py             # Custom loss functions
|   |   |-- metrics.py            # Evaluation metrics
|   |
|   |-- serving/
|   |   |-- __init__.py
|   |   |-- candidate_gen.py      # Candidate generation
|   |   |-- ranker.py             # Ranking service
|   |   |-- api.py                # FastAPI endpoints
|   |
```

```
|   |-- pipelines/
|       |-- __init__.py
|       |-- batch_pipeline.py    # Daily batch processing
|       |-- training_pipeline.py # Model training pipeline
|
|-- tests/
|   |-- unit/
|   |-- integration/
|   |-- fixtures/
|
|-- notebooks/
|   |-- 01_data_exploration.ipynb
|   |-- 02_feature_analysis.ipynb
|   |-- 03_model_experiments.ipynb
|
|-- scripts/
|   |-- train.py
|   |-- evaluate.py
|   |-- serve.py
|
|-- deployments/
    |-- kubernetes/
    |-- terraform/
```

B.2 Volume Guide

Volume	Title	Key Topics
1	Foundations, Architecture & Data Engineering	System design, tech stack, data schemas, pipelines
2	Model Development & Training	Two-tower, ranking, multi-objective, cold start
3	Serving, Deployment & Operations	Model serving, A/B testing, monitoring, MLOps
4	Advanced Topics & Interview Preparation	Advanced models, system design interviews, case studies

This concludes Volume 1 of the YouTube Shorts Recommendation System Production Guide. Volume 2 will cover model architecture implementation, training strategies, and evaluation methodologies in detail.

End of Volume 1